TraceBack — Code Coverage Report

6 Nov 2026 · companion to the TraceBack Test Reports

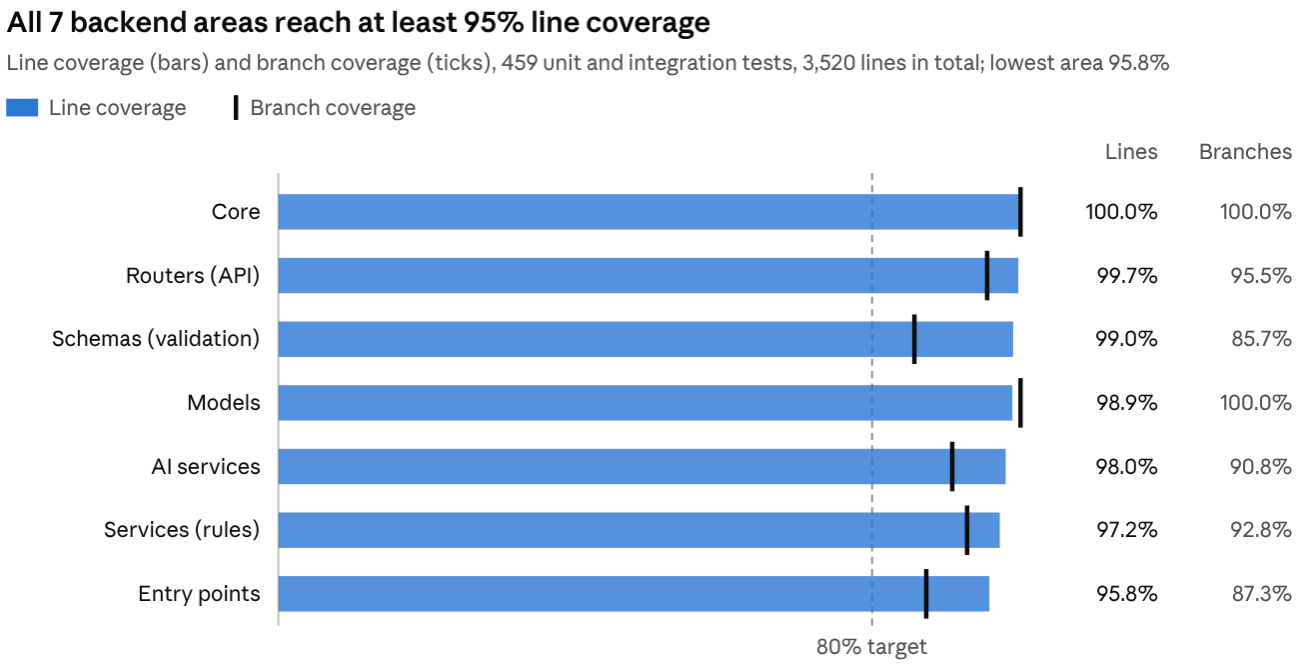
Result: the 459 automated tests run 97.9% of the backend's lines (3,445 of 3,520) and 91.2% of its decision branches (715 of 784). Every one of the 57 backend files has at least 91% line coverage, and 38 have every line covered.

TraceBack is the IIT Kanpur Lost & Found portal. This report measures how much of its Python backend the automated unit and integration tests run, and names what they still leave out.

How it was measured

- **Tool:** Coverage.py 7.17 through pytest-cov, measuring both lines and branches (each side of an `if`): `pytest --cov=app --cov-branch`.
- **Tests:** all 459 tests in `backend/tests`, all passing (JUnit file: `testing/results/backend_pytest_junit.xml`). The suite was run several times in a row with the same result, and the newest test files also in random order.
- **Real behaviour, no shortcuts:** no file is excluded from the measurement, and no line is marked "don't count". Every test checks an outcome (what is stored, sent, printed or refused), not just that the code runs. Only outside services are replaced by fakes that record what would be sent: Groq, Gemini, the Gmail SMTP server, the OCR model and the embedding model.
- **Data safety:** each test runs inside a database transaction that is undone at the end. Code that opens its own database sessions (the command-line tool, the worker, the seed script, the email log) joins that same transaction during tests, so no test leaves data behind.
- **Setup:** Python 3.14 and PostgreSQL 16, on a copy of the code. The browser system tests and adversarial tests also run the backend, but are not counted here; these figures come from `pytest` alone.

Coverage by area



coverage.py 7.17 with branch measurement · 459 unit and integration tests (pytest), 6 Nov 2026 · testing/results/coverage_by_area.json

Tests that cover the entry points and outside services

The command-line tool, the worker, the seed script, the AI clients and email delivery run outside the normal request path, so they have their own test files.

- **`test_cli.py`** (53 tests; `cli.py` 94.9%): every server command. `make-admin` creates or promotes the admin and refuses other email domains; `demo-items` and `ai-reindex` queue only active reports; `email-log` filters by item and address; `item-check` gives the right reason why an owner email was or wasn't sent at each stage; `email-test`, `ai-check`, `ai-run` and `ocr-check` report what they find; and each command reads its arguments correctly.
- **`test_worker_and_seed.py`** (14 tests; `worker.py` 97.6%, `seed.py` 96.6%, `services/ai/runner.py` 100%): the seed adds only what is missing; the worker runs all six jobs, sends a due reminder exactly once, keeps its schedule, survives a failing AI round and stops on Ctrl+C; the AI runner works through the queue, survives errors and stops cleanly.
- **`test_ai_clients.py`** (41 tests; `llm.py`, `vision.py`, `embeddings.py` and `semantic.py` 100%, `idcard.py` 99.4%): Groq first and Gemini as fallback, exactly what is sent to them, the error when both fail, the daily AI limit, the photo-description client, choosing and falling back between embedders, loading the OCR model, and the wording of match reasons.

- **`test_email_delivery.py`** (12 tests; `services/email.py` 100%): over SMTP, encryption starts before the password is sent, the From address is right, and mail-server failures are reported and logged; the email log uses its own session, cuts long values, never stops an email and hides sign-in codes; the console preview is saved.
- **`test_api_edges.py`** (13 tests; `routers/health.py`, `core/database.py`, `main.py` and `routers/reference.py` 100%): the health check with the database up and down, sessions always closed, start-up and shut-down, signing out without a session, proof images only through their own claim, a missing proof file, tidying of report fields, and the office list.
- **`test_hardening.py`** (25 tests): photo metadata is removed on upload (GPS gone, picture and rotation kept, damaged files refused), every API response carries the security headers, the API docs switch off, temporary mail-server failures are retried and wrong passwords are not, office emails never show placeholder text, and the AI agent follows its settings.
- **`test_failure_recovery.py`** (10 tests): with the database unreachable the API answers 503 at once; an AI job interrupted by a stopped worker runs again, up to 3 tries; a new report and its AI job are saved in one commit; a sign-in code that could not be emailed doesn't block a retry.

Least-covered files

File	Lines	Line coverage	Branch coverage	What is not run
`models/category.py`	12	91.7%	–	The debug text (`__repr__`), only seen in a Python console
`services/storage.py`	144	92.4%	81.5%	Rare damaged-file paths of the metadata stripper (a JPEG whose segments are cut short), an empty upload, and colour conversion for unusual image modes
`services/matching.py`	83	92.8%	94.4%	The error for an unknown `MATCHING_BACKEND` setting, and two failures that are logged and ignored

File	Lines	Line coverage	Branch coverage	What is not run
`services/ai/agent.py`	210	92.9%	83.3%	Safety checks for malformed answers from the language model: bad dates, missing ids, unknown tool names
`cli.py`	315	94.9%	86.8%	In `ai-run`: reading an ID card's photos and printing the steps of a successful AI agent run
`models/office.py`	22	95.5%	–	The debug text (`__repr__`)
`services/claims.py`	197	95.9%	92.9%	A duplicate claim caught by the database rule (run by concurrency test CON-05) and the 3-proof limit message

The gaps that remain are mostly defensive code for situations that are hard to cause on purpose, such as a language model answering in the wrong format. A fake agent run in `test_ai_agent.py` that sends malformed tool calls would cover most of what `agent.py` misses.

Frontend

The Next.js code has no automated unit tests, so its coverage was not measured. It is exercised by the 15 browser system tests and the XSS check, on 15 of the site's 18 pages. Component tests, or browser coverage collected during the system tests, would close this gap.

Full report and how to reproduce

The line-by-line report is in `testing/results/coverage_html/`. Open `index.html` in a browser: covered lines are green, missed lines red and partly-run branches yellow. The raw figures are in `testing/results/backend_coverage.json`, with area and per-file summaries in `coverage_by_area.json` and `coverage_by_file.json`.

To measure again: `cd backend`, then `pytest --cov=app --cov-branch --cov-report=html` (needs `pytest-cov`). The report appears in `backend/htmlcov/index.html`.